ASSIGNMENT 18
SQL Injection Detection and Prevention Using Parameterized Queries

1. Introduction

SQL Injection is a vulnerability that occurs when an application inserts untrusted input directly into an SQL statement. In Android applications, unsafe database operations can allow input to change query logic, expose records, or modify data. This practical explains how to review database operations for unsafe query construction and how parameterized queries help prevent injection.

2. Objectives

- Identify database operations that concatenate user input into SQL statements.
- Understand the security risks associated with unsafe queries.
- Demonstrate the use of parameterized queries in Android SQLite operations.
- Compare expected behavior before and after applying the secure fix.

3. Requirements

- Android Studio and a local Android test application.
- An emulator or test device.
- SQLite or the Android SQLiteDatabase API.
- A test database populated only with dummy records.
- Permission to inspect and modify the application being tested.

Carry out the exercise only in a local application and database you own or are authorized to assess. Do not test production applications or use real user data.

4. Theory

A vulnerable database operation may construct SQL by joining a query string with a username, search term, or other user-supplied value. The database may interpret special characters as SQL syntax rather than as ordinary data. This can cause unintended results or changes.

Parameterized queries separate the SQL statement from its values. Placeholders such as ? represent values, and the database binds the supplied values separately. As a result, input is treated as data and cannot alter the query structure.

5. Procedure

Step 1: Prepare a local test database

Create a small test table, such as users or products, and insert a few dummy records. Use synthetic data only. Identify the Android methods that query, insert, update, or delete records based on user input.

Step 2: Inspect database operations

Search the code for rawQuery(), execSQL(), SQLiteDatabase query methods, and SQL strings assembled using + or string formatting. Check whether user-controlled values are directly inserted into SQL text. Record the method name and the type of operation.

Step 3: Recognize an unsafe query

The following example illustrates unsafe string concatenation for a search operation:

```
String sql = "SELECT * FROM products WHERE name = '" + searchText + "'";
Cursor c = db.rawQuery(sql, null);
```

Because searchText is inserted into the SQL statement itself, specially formed input may change the query's meaning. Avoid copying this pattern into production code.

Step 4: Replace it with a parameterized query

Use a placeholder and pass the user value separately:

```
Cursor c = db.rawQuery("SELECT * FROM products WHERE name = ?",
    new String[] { searchText });
```

Alternatively, use Android's query() method with selection arguments:

```
Cursor c = db.query("products", null, "name = ?",
    new String[] { searchText }, null, null, null);
```

These examples demonstrate parameter binding. Ensure that the cursor is closed when no longer needed, preferably with try/finally or a suitable resource-management pattern.

Step 5: Apply the same principle to other operations

For inserts, updates, and deletes, use methods that accept ContentValues or bound arguments. Never build a query by concatenating untrusted input. SQL identifiers such as table or column names generally cannot be supplied as ordinary value placeholders; if dynamic identifiers are required, choose them from a strict allowlist.

Step 6: Test before and after the fix

Use normal search input and a controlled test string containing SQL punctuation in the local dummy database. Before the fix, observe whether unsafe query construction changes the returned results or produces an SQL error. After the fix, verify that the same input is treated as a literal value and does not change the SQL structure. Do not test against external systems.

6. Analysis

The root cause is the mixing of SQL syntax and untrusted data. Parameterized queries ensure the database parses the SQL structure separately from the supplied values. They are the primary defense for SQL values, but they do not replace authorization checks, safe error handling, or input validation required by the application's business rules.

For example, if a user searches for a product name, the bound value is matched as a value. SQL punctuation in the search text is not executed as part of the query. If a database action involves access to protected data, authorization must also be checked independently.

7. Observation Table

Test / Code pattern	Expected behavior	Unsafe implementation	Parameterized implementation
Normal search value	Return matching record	Usually returns expected result	Returns matching record
Input containing SQL punctuation	Treat input as a literal value	May change query meaning or cause an error	Treated as a bound value
No matching record	Return no rows	May return unexpected rows if query is altered	Returns no matching rows
Insert/update/delete with user data	Change only intended record	Concatenation may permit unintended query behavior	Use ContentValues or bound values

8. Result

Unsafe SQL construction was identified as a potential SQL Injection risk, and parameterized-query examples were applied to separate SQL statements from user-provided values. In the local test environment, the expected secure behavior is that input is treated as data and cannot change the query structure. Record actual observations from the application tested.

9. Precautions

- Use only an authorized local test application and dummy records.
- Do not run tests against production or third-party systems without explicit permission.
- Use placeholders for SQL values and never concatenate untrusted input into SQL statements.
- Use allowlists for dynamic table or column identifiers when unavoidable.
- Close database cursors and handle errors without exposing sensitive database details.

10. Conclusion

SQL Injection risks in Android database operations can be reduced by identifying unsafe string concatenation and replacing it with parameterized queries or appropriate SQLiteDatabase APIs. Parameter binding keeps user input separate from SQL syntax and helps prevent malicious input from altering query logic. Secure database access should be combined with proper authorization, safe error handling, and careful data management.

11. References

- Android Developers documentation for SQLiteDatabase and database access.
- OWASP SQL Injection Prevention Cheat Sheet.
- OWASP Mobile Application Security Testing Guide (MASTG).